

7- Community Sports League Fixtures

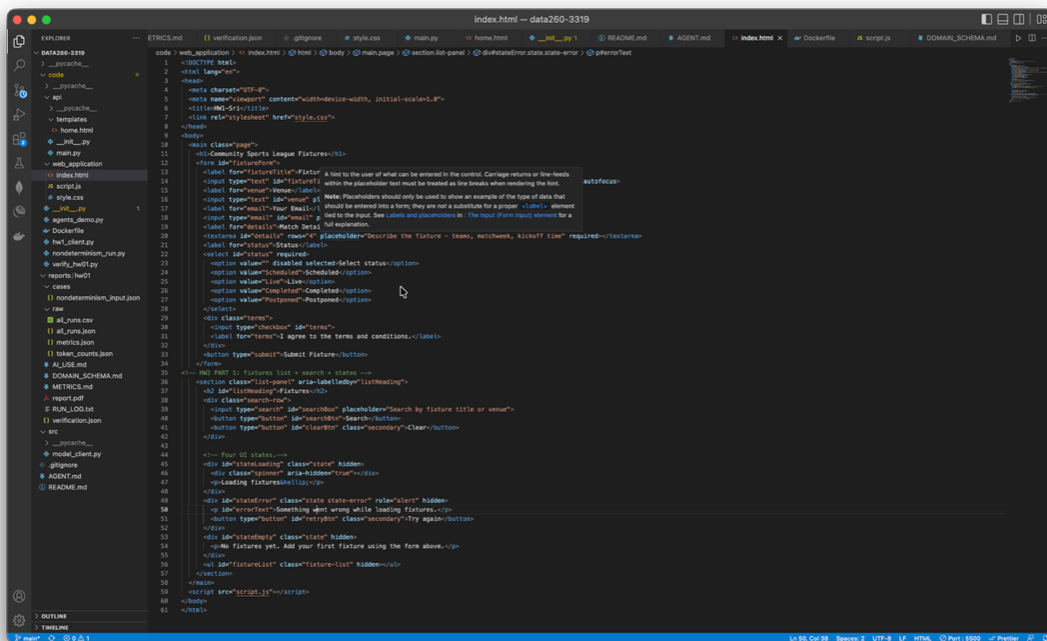
Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

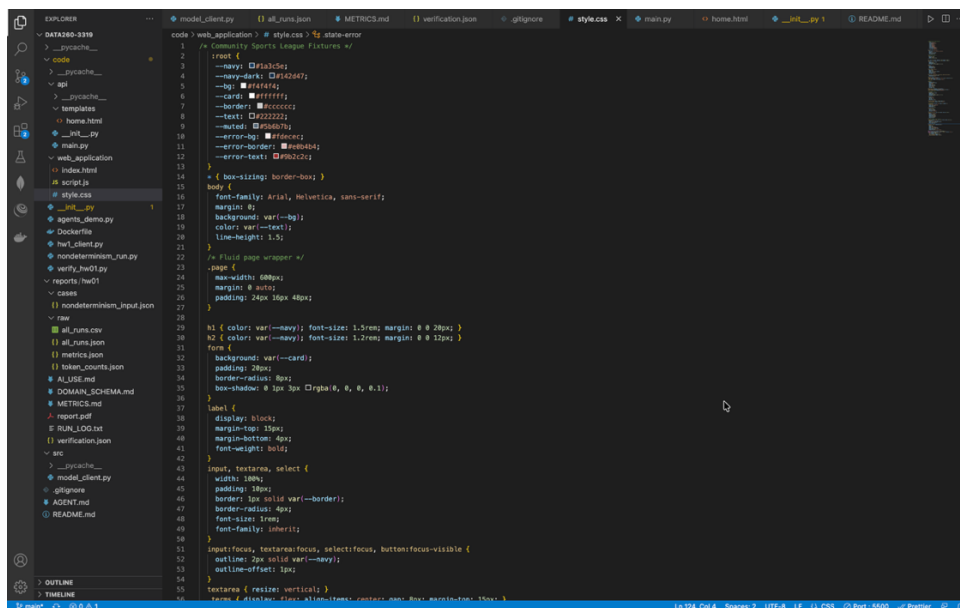
GitHub repository: <https://github.com/villyalsriprajeet-cmd/data260-3319>

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7

This part builds on my HW1 web application. I made the page work at 375px width, and added visible loading, empty and error states.

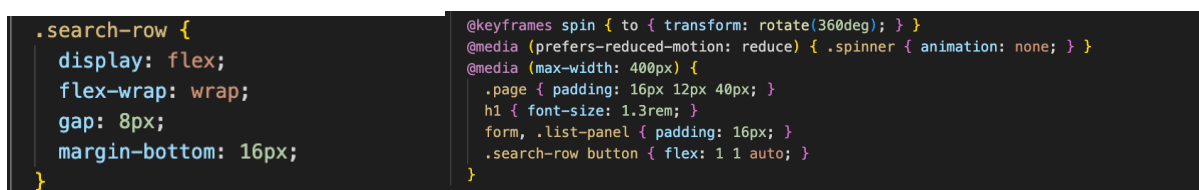


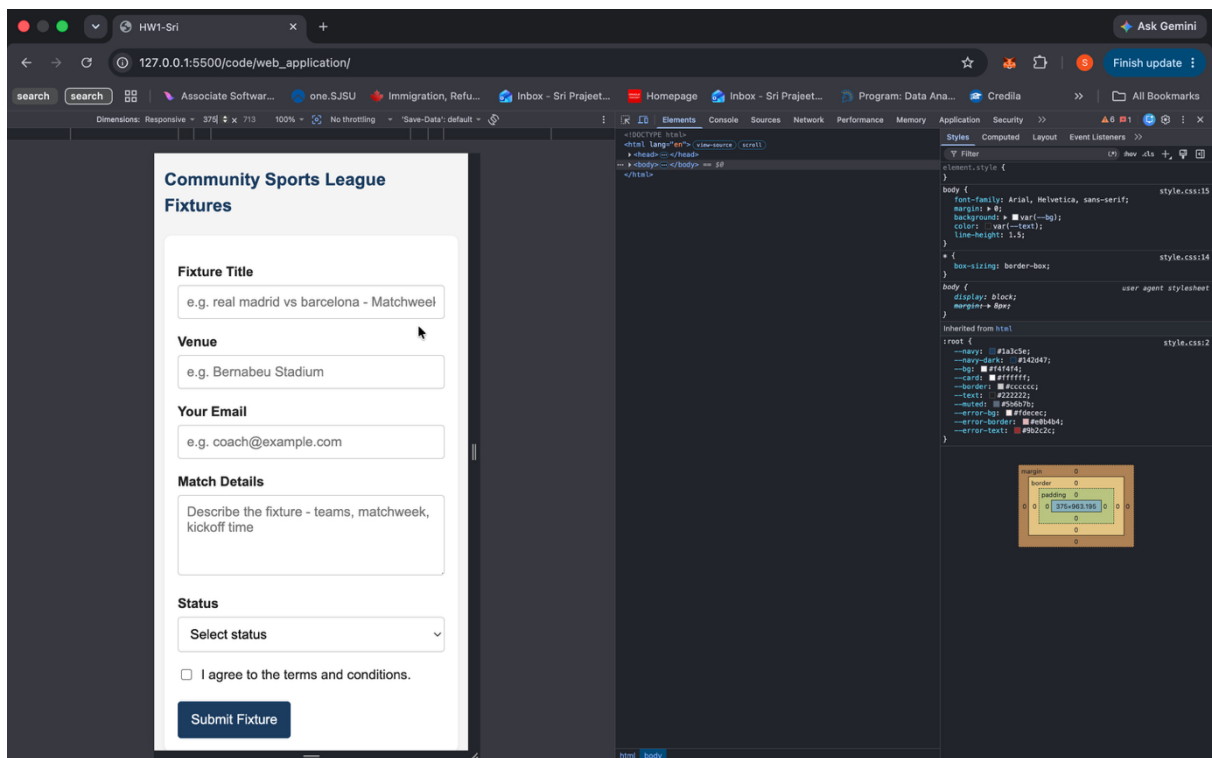
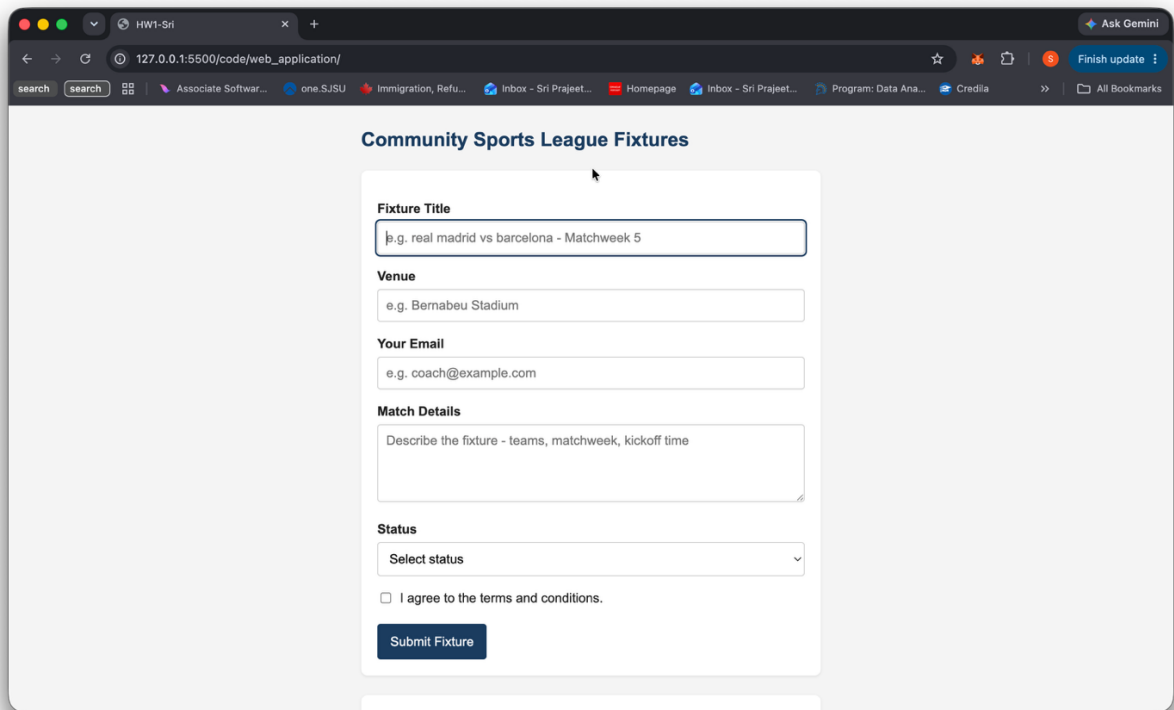


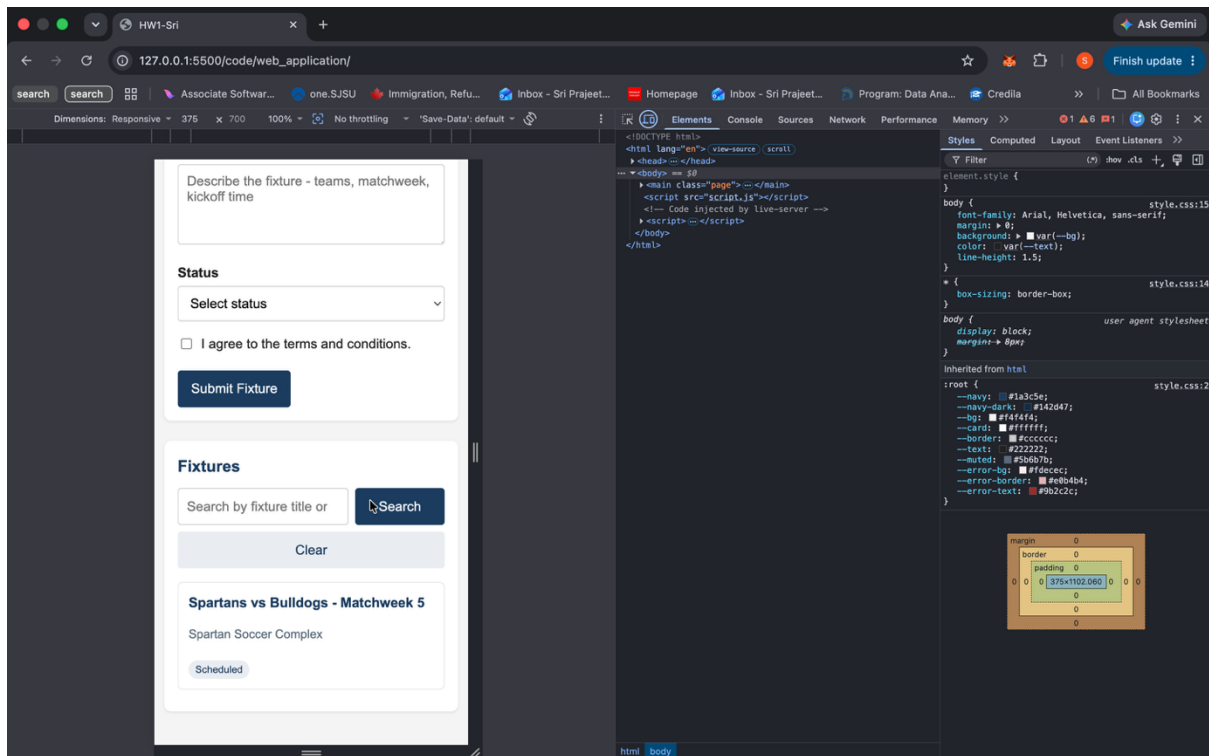
This wrapper is the main thing that makes 375px work, it caps the width on big screens but shrinks with side padding on a phone so nothing gets cut off.



Flex wrap lets the search box and buttons drop onto a new line instead of overflowing when the screen gets narrow. Media query tightens the padding and makes the buttons full width once they wrap.







Loading, empty, and error states

I added a small fixtures list and three state boxes in index.html. All three start out hidden, and the JavaScript shows exactly one at a time depending on what's happening.

```

<!-- Four UI states.-->
<div id="stateLoading" class="state" hidden>
  <div class="spinner" aria-hidden="true"></div>
  <p>Loading fixtures&hellip;</p>
</div>
<div id="stateError" class="state state-error" role="alert" hidden>
  <p id="errorText">Something went wrong while loading fixtures.</p>
  <button type="button" id="retryBtn" class="secondary">Try again</button>
</div>
<div id="stateEmpty" class="state" hidden>
  <p>No fixtures yet. Add your first fixture using the form above.</p>
</div>
<ul id="fixtureList" class="fixture-list" hidden></ul>
</section>
</main>
<script src="script.js"></script>
</body>
</html>

```

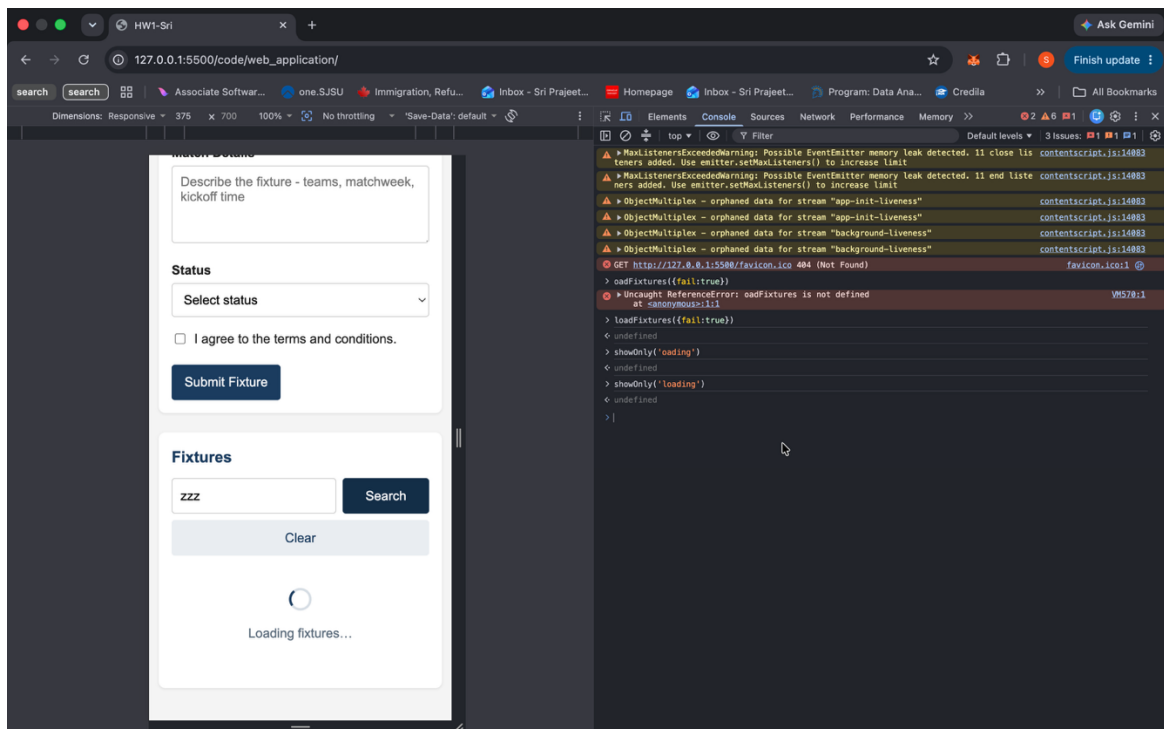
The switching is done by one small helper function. It just hides everything except the state I ask for, so only one is ever on screen.

```

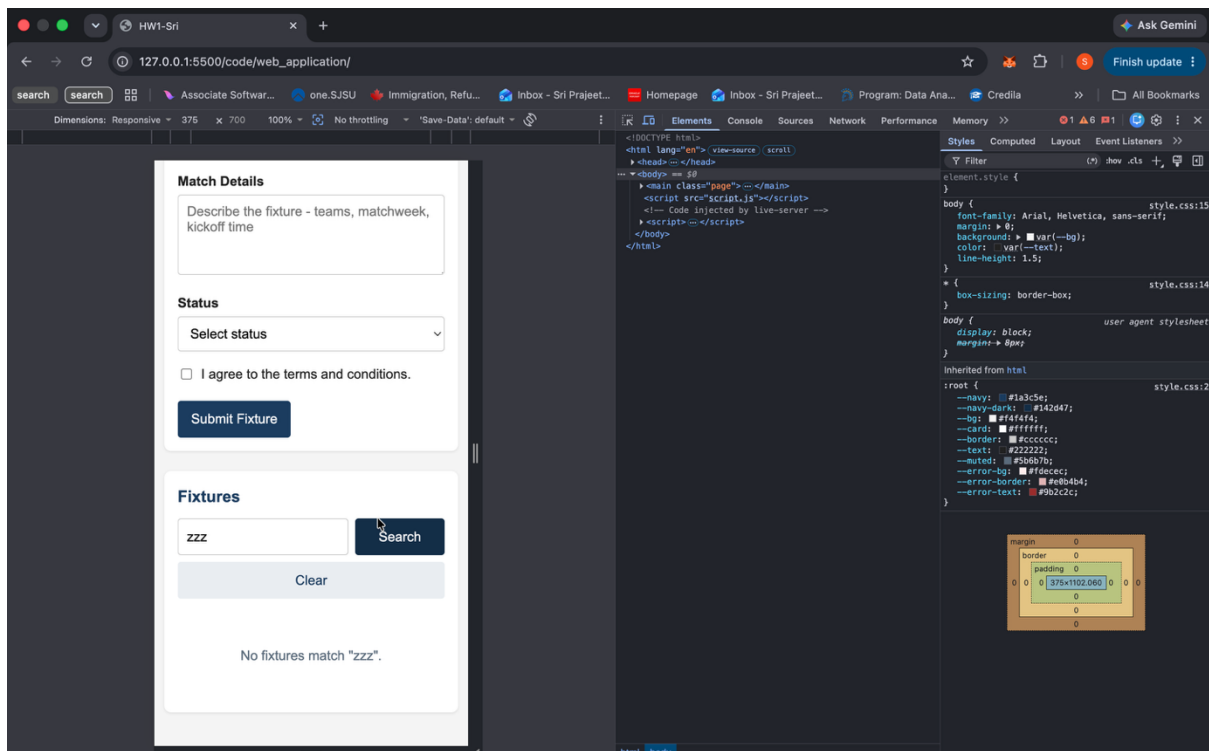
// visible loading, empty, and error states.
function showOnly(state) {
  els.loading.hidden = state !== "loading";
  els.error.hidden = state !== "error";
  els.empty.hidden = state !== "empty";
  els.list.hidden = state !== "data";
}

```

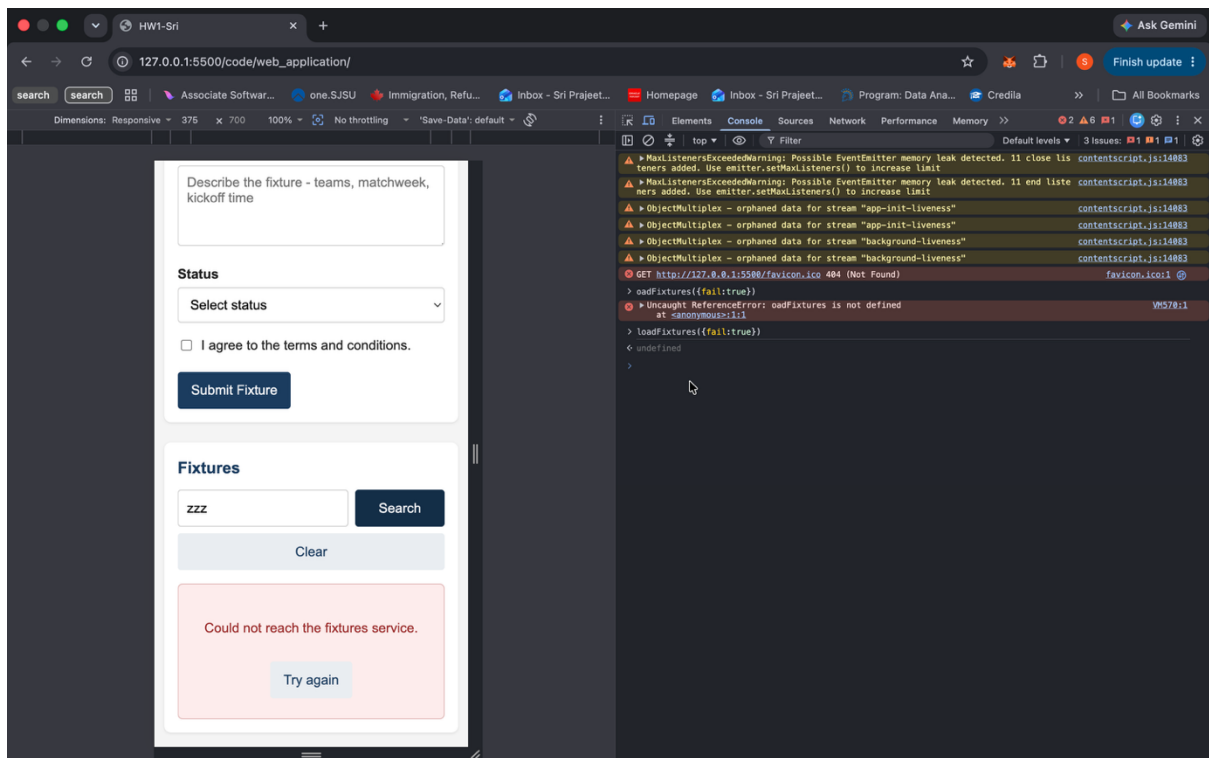
Loading state



Empty State

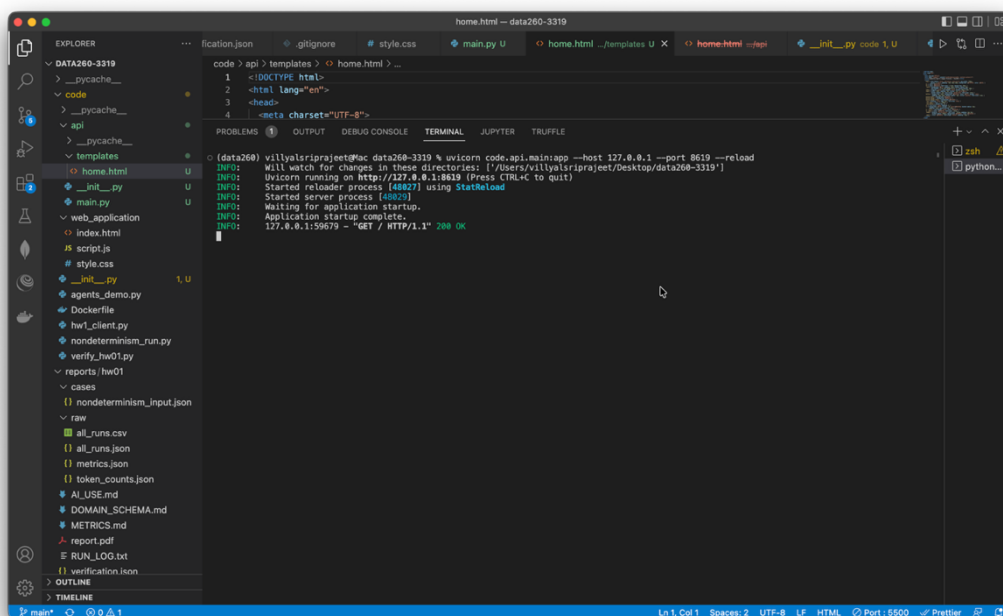


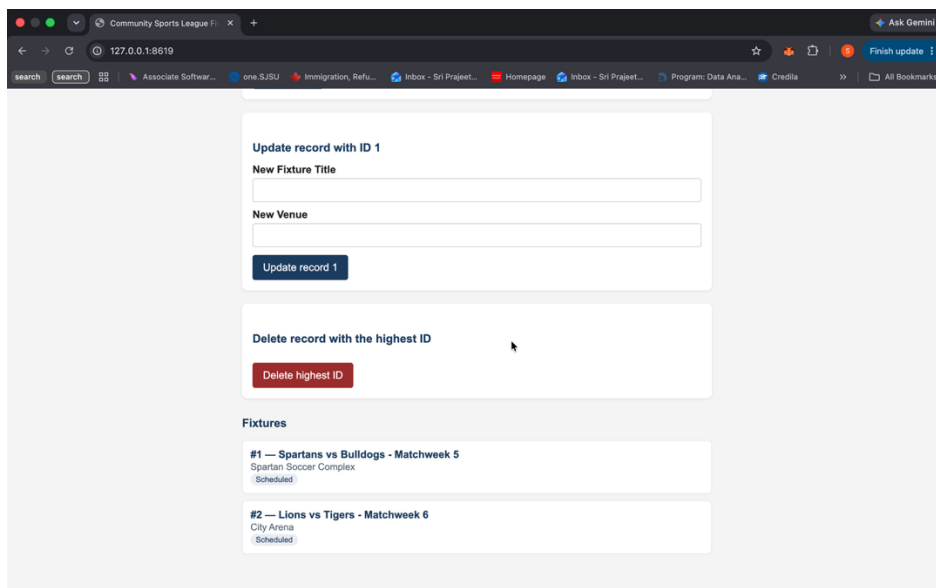
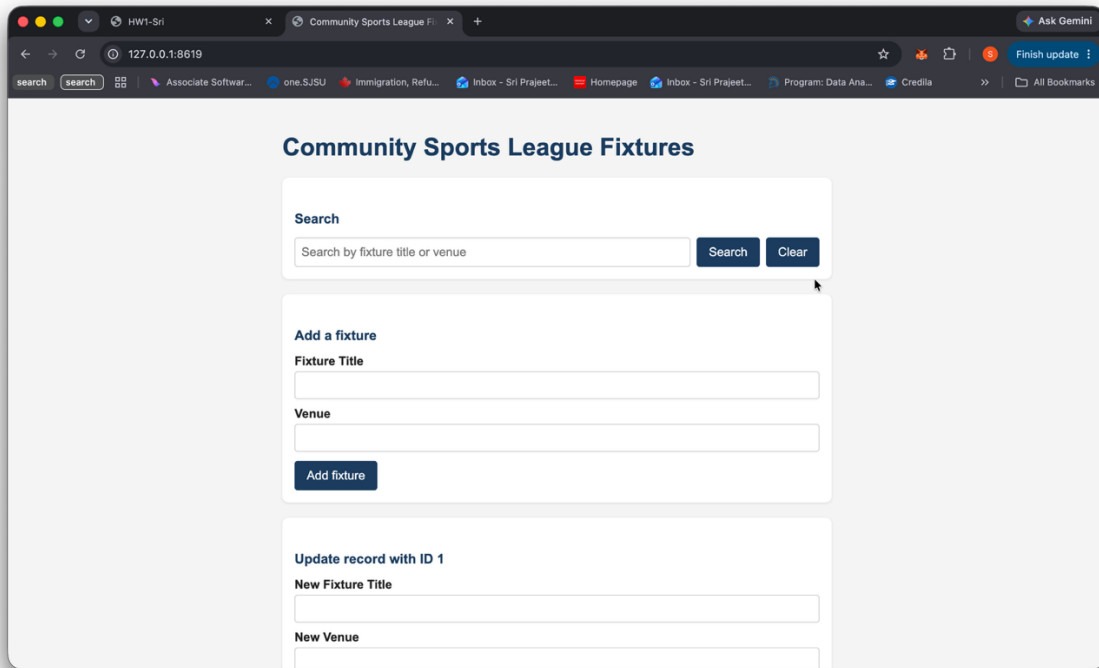
Error State



Part 2 FastAPI

Implemented a FastAPI backend running on my PORT_BASE (8619) that can add a record, update the record with ID 1, delete the record with the highest ID, and search by the primary or secondary field, and after each action it redirects back to the home view showing the updated list. For my domain the primary field is the Fixture Title and the secondary field is the Venue.





Created two files `main.py` with the routes and `home.html` that the server renders. The fixtures are held in a simple in-memory list, and each write action ends with a redirect back to the home page.

1. Add a record

The add form sends the Fixture Title and Venue. The route builds a new record, appends it to the list, and redirects home so the new one shows up straight away.

```
# 1. ADD A RECORD
@app.post("/add")
def add_fixture(fixtureTitle: str = Form(...), venue: str = Form(...)):
    fixtures.append({
        "id": get_next_id(),
        "fixtureTitle": fixtureTitle,
        "venue": venue,
        "status": "Scheduled",
    })
    return RedirectResponse(url="/", status_code=303)
```

The screenshot shows a web browser window with the URL 127.0.0.1:8619. The page has a search bar at the top with the text "Search by fixture title or venue" and buttons for "Search" and "Clear". Below the search bar, there are three main sections:

- Add a fixture**: This section contains a "Fixture Title" input field with the text "Eagles vs Hawks - Matchweek 7", a "Venue" input field with the text "Riverside Field", and an "Add fixture" button.
- Update record with ID 1**: This section contains a "New Fixture Title" input field, a "New Venue" input field, and an "Update record 1" button.
- Delete record with the highest ID**: This section is currently empty.

The screenshot shows the same web browser window, but now the "Add a fixture" form is filled out. The "Fixture Title" field contains "Eagles vs Hawks - Matchweek 7" and the "Venue" field contains "Riverside Field". The "Update record 1" button is still visible. Below the "Delete record with the highest ID" section, there is a "Fixtures" section that displays a list of three fixtures:

- #1 — Spartans vs Bulldogs - Matchweek 5**: Spartan Soccer Complex, Scheduled
- #2 — Lions vs Tigers - Matchweek 6**: City Arena, Scheduled
- #3 — Eagles vs Hawks - Matchweek 7**: Riverside Field, Scheduled

2. Update the record with ID 1

This route loops through the list, finds the record whose id is 1, overwrites its title and venue with whatever was submitted, and redirects home

```
# 2. UPDATE RECORD WITH ID 1
@app.post("/update/1")
def update_fixture_1(fixtureTitle: str = Form(...), venue: str = Form(...)):
    for f in fixtures:
        if f["id"] == 1:
            f["fixtureTitle"] = fixtureTitle
            f["venue"] = venue
            break
    return RedirectResponse(url="/", status_code=303)
```

Update record with ID 1

New Fixture Title
Spartans vs Wildcats - Final

New Venue
Championship Stadium

Update record 1

Delete record with the highest ID
Delete highest ID

Fixtures

- #1 — Spartans vs Wildcats - Final
Championship Stadium
Scheduled
- #2 — Lions vs Tigers - Matchweek 6
City Arena
Scheduled
- #3 — Eagles vs Hawks - Matchweek 7
Riverside Field
Scheduled

I changed record #1 to "Spartans vs Wildcats - Final" at "Championship Stadium". It updated in place and kept the same id of 1.

Update record with ID 1

New Fixture Title
Spartans vs Wildcats - Final

New Venue
Championship Stadium

Update record 1

Delete record with the highest ID
Delete highest ID

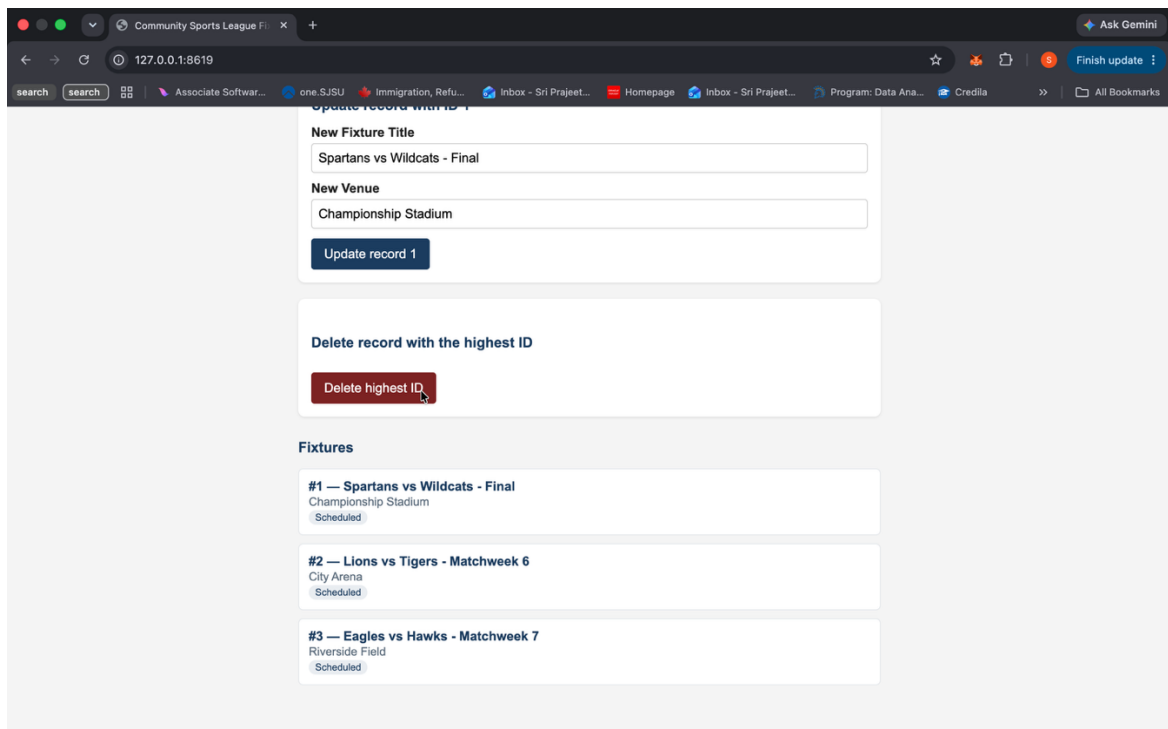
Fixtures

- #1 — Spartans vs Wildcats - Final
Championship Stadium
Scheduled
- #2 — Lions vs Tigers - Matchweek 6
City Arena
Scheduled
- #3 — Eagles vs Hawks - Matchweek 7
Riverside Field
Scheduled

3. Delete the record with the highest ID

This one finds the record with the largest id using `max()` and removes it, then redirects home.

```
# 3. DELETE THE RECORD WITH THE HIGHEST ID
@app.post("/delete-highest")
def delete_highest():
    if fixtures:
        highest = max(fixtures, key=lambda f: f["id"])
        fixtures.remove(highest)
        return RedirectResponse(url="/", status_code=303)
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=PORT_BASE)
```



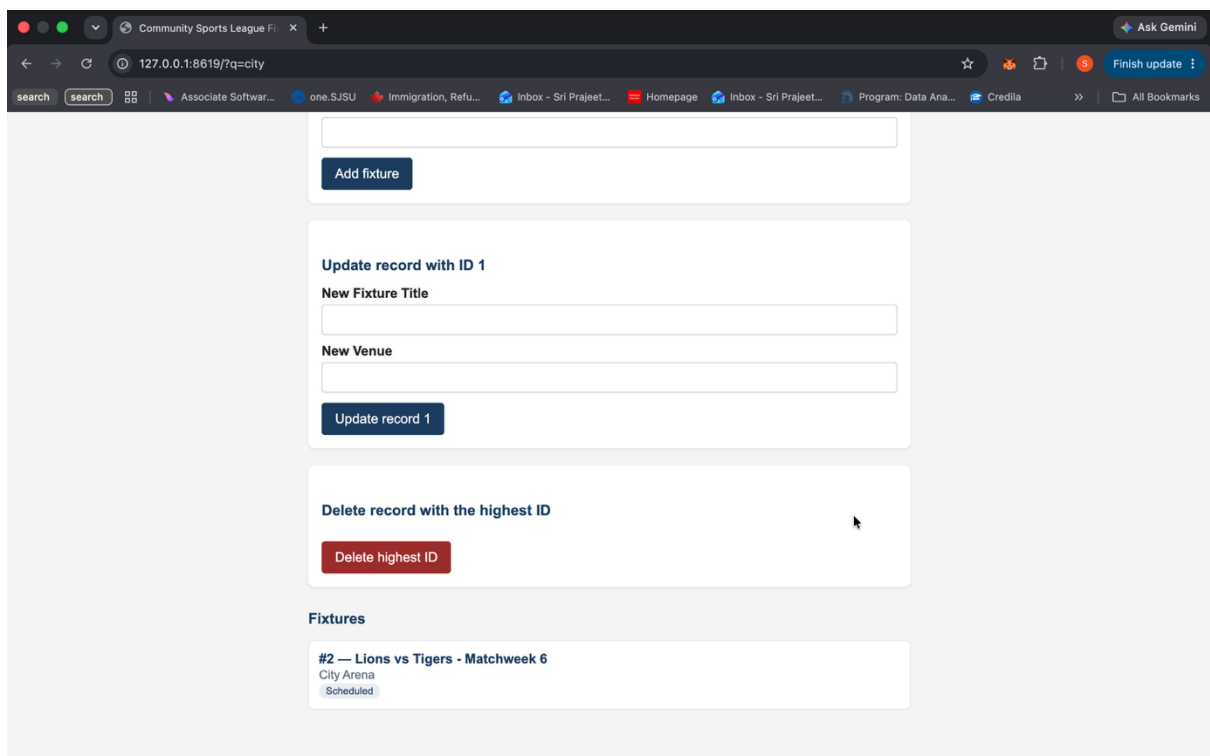
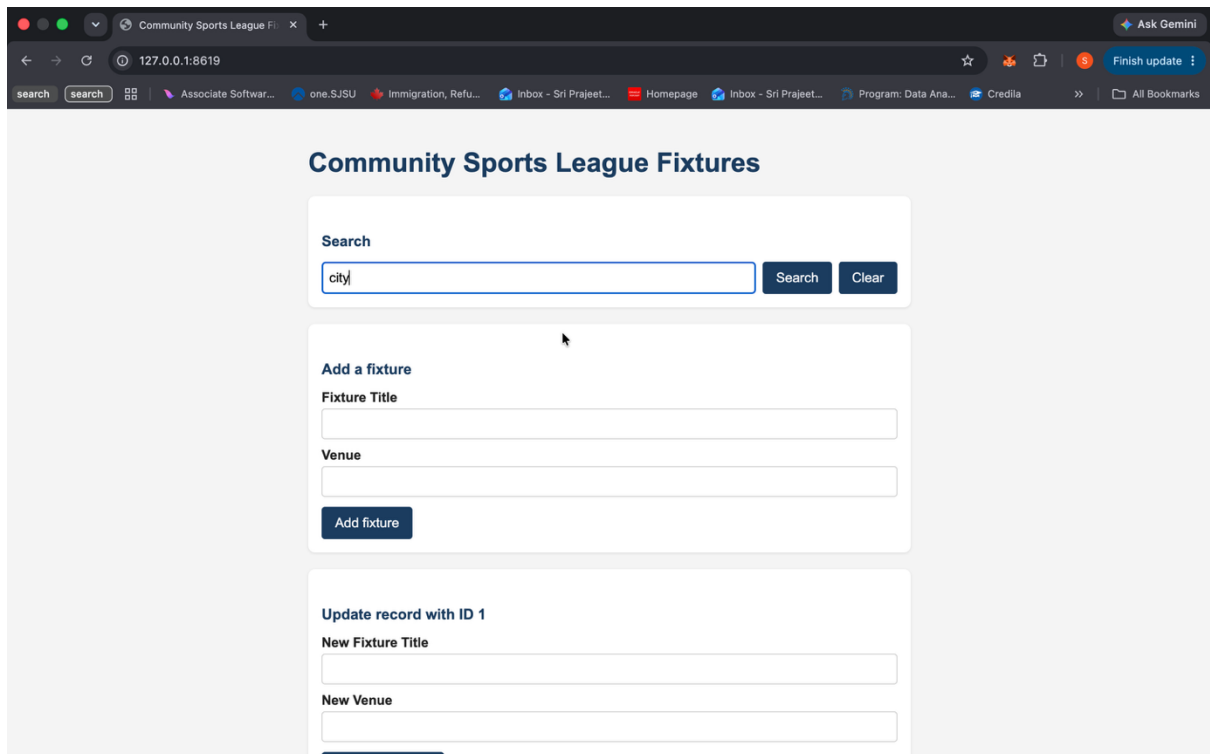
4. Search by title or venue

I handled search on the home route itself. If a search term comes in through the q query string, the list is filtered to only the fixtures whose title or venue contains that term. If there's no term, it shows everything.

```
# HOME VIEW (also handles search)
@app.get("/")
def home(request: Request, q: str = ""):
    query = q.strip().lower()
    if query:
        visible = [
            f for f in fixtures
            if query in f["fixtureTitle"].lower() or query in f["venue"].lower()
        ]
    else:
        visible = fixtures

    return templates.TemplateResponse(
        request,
        "home.html",
        {"fixtures": visible, "q": q},
    )
```

Searching "city" narrowed the list down to the single fixture at City Arena.



The Clear button brought the full list back.

Part 3 - Stateful Agent Graph

In HW1 the Planner and Reviewer ran one after another in a fixed order, a straight waterfall that always did the same thing and then stopped. For HW2 I rebuilt that as a stateful graph using LangGraph, so a supervisor can decide where to send the work and the graph can loop back on itself to self correct. I put this in a new file, code/agents_graph.py. All the LLM calls in the graph go through the same src/model_client.py adapter, not through ChatOllama.

Step 2: Setting up the State

I defined the shared memory that every node reads from and writes to. It holds the inputs, a slot for each agent's output, and a turn counter that stops the graph from looping forever.

```
# Step 2 – AgentState: the shared memory for every node
class AgentState(TypedDict):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    planner_proposal: Dict[str, Any]
    reviewer_feedback: Dict[str, Any]
    turn_count: int
```

Step 3: Creating the Agent Nodes

Each node is just a function that takes the state, does its job, and returns a dictionary with only the keys it wants to update. The important part is that both nodes call the model through the ModelClient, so the actual LLM request goes through `model_client.py` and nowhere else.

```
def _call_llm(state: AgentState, system_prompt: str, extra_context: str = "") -> Dict[str, Any]:
    llm: ModelClient = state["llm"]
    user = (
        f"Task:\n{state['task']}\n\n"
        f"{extra_context}\n"
        "Reply with ONE JSON object only."
    )
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user},
    ]
    result = llm.complete(messages)
    return parse_and_coerce(
        result["text"], state["title"], state["content"], state["strict"]
    )

def planner_node(state: AgentState) -> Dict[str, Any]:
    print(["NODE: Planner"])
    proposal = _call_llm(state, PLANNER_SYSTEM)
    return {"planner_proposal": proposal}

def reviewer_node(state: AgentState) -> Dict[str, Any]:
    print("NODE: Reviewer")
    proposal = state.get("planner_proposal", {})
    context = "The Planner proposed:\n" + json.dumps(proposal.get("data", {}), indent=2)
    feedback = _call_llm(state, REVIEWER_SYSTEM, extra_context=context)
    # feedback["data"]["issues"] = ["forced issue for loop demo"]
    return {"reviewer_feedback": feedback}
```

Step 4: Building the Supervisor (The Router)

The supervisor is split into two parts. The supervisor node only updates the state and the router just reads the state and returns a string saying where to go next.

```
def supervisor_node(state: AgentState) -> Dict[str, Any]:
    print("NODE: Supervisor")
    return {"turn_count": state.get("turn_count", 0) + 1}

def router_logic(state: AgentState) -> str:
    if not state.get("planner_proposal"):
        return "planner"

    feedback = state.get("reviewer_feedback") or {}
    issues = feedback.get("data", {}).get("issues", [])

    if not feedback:
        return "reviewer"

    if issues and state.get("turn_count", 0) < TURN_CEILING:
        return "planner"

    return END
```

So if there's no proposal yet it goes to the Planner, if there's a proposal but no review yet it goes to the Reviewer, and if the Reviewer found problems and we're still under the turn ceiling it loops back to the Planner to try again. Otherwise it ends. I set `TURN_CEILING = 6` as the safety limit so the correction loop can't run forever.

Step 5: Assembling the Graph

The graph always starts at the supervisor, the supervisor uses the router to pick the next node, and both workers hand control back to the supervisor when they're done, which is what creates the loop.

```
def build_graph():
    graph = StateGraph(AgentState)

    graph.add_node("supervisor", supervisor_node)
    graph.add_node("planner", planner_node)
    graph.add_node("reviewer", reviewer_node)

    graph.add_edge(START, "supervisor")

    graph.add_conditional_edges(
        "supervisor",
        router_logic,
        {"planner": "planner", "reviewer": "reviewer", END: END},
    )

    graph.add_edge("planner", "supervisor")
    graph.add_edge("reviewer", "supervisor")

    return graph.compile()
```

Step 6: Running and Testing

I built the initial state and ran the graph with `.stream()` so I could see the output from each step as it happened, instead of just the final answer.

When I run it normally, the graph goes Supervisor -> Planner -> Supervisor -> Reviewer -> Supervisor -> end. The Reviewer doesn't find any problems, so even though the ceiling is 6 it stops early at turn 3, which is exactly what should happen on a clean run.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL JUPYTER TRUFFLE
(data260) villyalsriprajeet@Mac data260-3319 % python code/agents_graph.py
NODE: Supervisor
[Supervisor] -> {"turn_count": 1}
NODE: Planner
[planner] -> {"planner_proposal": {"thought": "I'll extract key information and create concise tags and a summary.", "message": "San Jose State vs Fresno State soccer match details.", "data": {"tags": ["spartansbulldogs", "mountainwestsoccer", "state"], "summary": "San Jose State and Fresno State face off in a M
NODE: Supervisor
[Supervisor] -> {"turn_count": 2}
NODE: Reviewer
[reviewer] -> {"reviewer_feedback": {"thought": "", "message": "Tags and summary prepared.", "data": {"tags": ["state", "soccer", "fresno state bulldogs"], "summary": "San Jose Stat
e Spartans vs Fresno State Bulldogs - Mountain West Soccer Fixture. The San Jose State Spartans host the Fresno State Bulldogs in a."
NODE: Supervisor
[Supervisor] -> {"turn_count": 3}

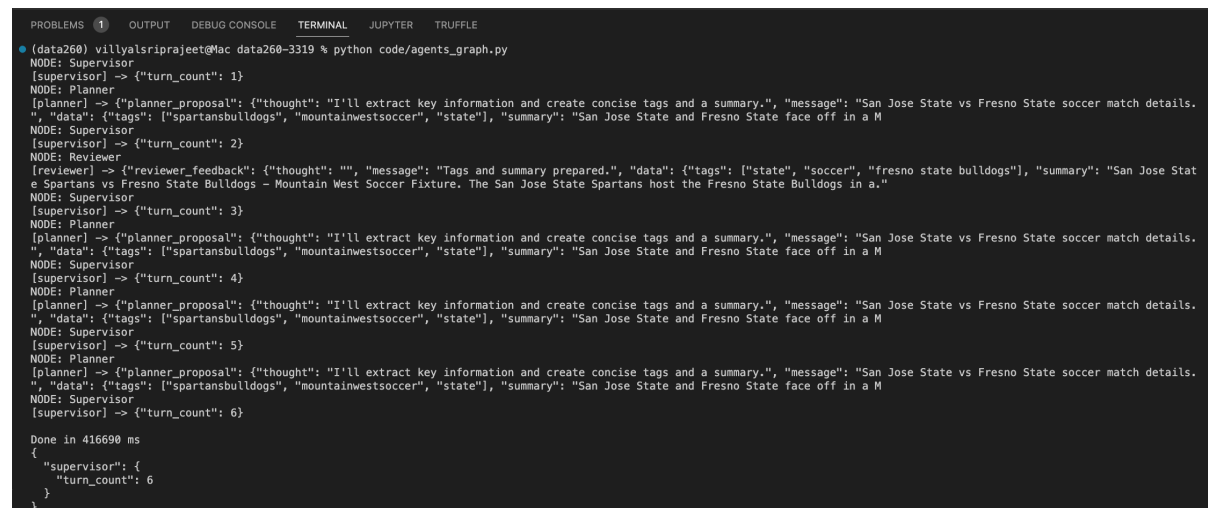
Done in 147650 ms
{
  "supervisor": {
    "turn_count": 3
  }
}
(data260) villyalsriprajeet@Mac data260-3319 %
```

Testing the self correction loop

To test the correction loop, I used the line `feedback["data"]["issues"] = ["forced issue for loop demo"]` to make the Reviewer always report a problem. Once it always finds an issue, the router keeps routing back to the Planner, so the Planner runs again and again until the turn count reaches the ceiling of 6 and the graph ends.

```
def reviewer_node(state: AgentState) -> Dict[str, Any]:
    print("NODE: Reviewer")
    proposal = state.get("planner_proposal", {})
    context = "The Planner proposed:\n" + json.dumps(proposal.get("data", {}), indent=2)
    feedback = _call_llm(state, REVIEWER_SYSTEM, extra_context=context)
    feedback["data"]["issues"] = ["forced issue for loop demo"]
    return {"reviewer_feedback": feedback}
```

```
feedback["data"]["issues"] = ["forced issue for loop demo"]
```



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL JUPYTER TRUFFLE
• (data260) villyalsriprajeet@mac data260-3319 % python code/agents_graph.py
NODE: Supervisor
[Supervisor] -> {"turn_count": 1}
NODE: Planner
[Planner] -> {"planner_proposal": {"thought": "I'll extract key information and create concise tags and a summary.", "message": "San Jose State vs Fresno State soccer match details.", "data": {"tags": ["spartansbulldogs", "mountainwestsoccer", "state"], "summary": "San Jose State and Fresno State face off in a M
NODE: Supervisor
[Supervisor] -> {"turn_count": 2}
NODE: Reviewer
[Reviewer] -> {"reviewer_feedback": {"thought": "", "message": "Tags and summary prepared.", "data": {"tags": ["state", "soccer", "fresno state bulldogs"], "summary": "San Jose Stat
e Spartans vs Fresno State Bulldogs - Mountain West Soccer Fixture. The San Jose State Spartans host the Fresno State Bulldogs in a."
NODE: Supervisor
[Supervisor] -> {"turn_count": 3}
NODE: Planner
[Planner] -> {"planner_proposal": {"thought": "I'll extract key information and create concise tags and a summary.", "message": "San Jose State vs Fresno State soccer match details.", "data": {"tags": ["spartansbulldogs", "mountainwestsoccer", "state"], "summary": "San Jose State and Fresno State face off in a M
NODE: Supervisor
[Supervisor] -> {"turn_count": 4}
NODE: Planner
[Planner] -> {"planner_proposal": {"thought": "I'll extract key information and create concise tags and a summary.", "message": "San Jose State vs Fresno State soccer match details.", "data": {"tags": ["spartansbulldogs", "mountainwestsoccer", "state"], "summary": "San Jose State and Fresno State face off in a M
NODE: Supervisor
[Supervisor] -> {"turn_count": 5}
NODE: Planner
[Planner] -> {"planner_proposal": {"thought": "I'll extract key information and create concise tags and a summary.", "message": "San Jose State vs Fresno State soccer match details.", "data": {"tags": ["spartansbulldogs", "mountainwestsoccer", "state"], "summary": "San Jose State and Fresno State face off in a M
NODE: Supervisor
[Supervisor] -> {"turn_count": 6}

Done in 416690 ms
{
  "supervisor": {
    "turn_count": 6
  }
}
```

Note :

The runs were slow at first, so I updated `model_client.py` to pass Ollama a smaller context window and output limit (`num_ctx=512`, `num_predict=256`) and to ask for JSON output (`format="json"`), the same settings my HW1 script used. I added them with defaults so nothing else that uses the client breaks. That brought each run down.

Part 4 - Output Schema and Loop Safety

I checked the Planner's output against a Pydantic schema, and if it doesn't pass I send the error back to the Planner and let it try again. Then I ran three experiments on it. Everything ran on `qwen2.5:3b` locally through Ollama.

The schema

The schema is in `code/schema_hw02.py`. It wants exactly 3 tags, each 3 to 30 characters, and a summary of 25 words or fewer.


```

class PlannerOutput(BaseModel):
    tags: List[str]
    summary: str
    @field_validator("tags")
    @classmethod
    def check_tags(cls, v):
        if len(v) != 3:
            raise ValueError(f"must have exactly 3 tags, got {len(v)}")
        for t in v:
            if not isinstance(t, str):
                raise ValueError("every tag must be a string")
            if not (3 <= len(t) <= 30):
                raise ValueError(f"tag '{t}' must be 3-30 characters, got {len(t)}")
        return v
    @field_validator("summary")
    @classmethod
    def check_summary(cls, v):
        n = len(v.split())
        if n > 25:
            raise ValueError(f"summary must be at most 25 words, got {n}")
        return v

```

Validation and retry

The runner (code/runner_hw02.py) calls the Planner, validates the output, and if it fails it puts the error into the next prompt and tries again, up to the turn ceiling. It records whether the run was valid, how many retries it took, and how long it took.

```

for attempt in range(turn_ceiling):
    proposal = _planner_call(llm, task, title, content, strict, error_feedback)
    data = proposal.get("data", {})
    ok, err = validate_planner({"tags": data.get("tags", []),
                               "summary": data.get("summary", "")})
    if ok:
        return {
            "valid": True,
            "retries": retries,
            "latency_ms": int((time.time() - t0) * 1000),
            "final": data,
        }
    error_feedback = err
    retries += 1

return {
    "valid": False,
    "retries": retries,
    "latency_ms": int((time.time() - t0) * 1000),
    "final": (proposal or {}).get("data", {}),
}

```

Experiment 1 - 30 runs on a fixed input

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER: VARIABLES

Experiment 1 - 30-run schema test
runs: 30 completion rate: 100.00% overall mean latency: 117420 ms

Outcome over 30 runs      Count  Mean latency (ms)
Valid first attempt      30      117420
Valid after 1 retry       0        -
Valid after 2+ retries    0        -
Hit turn ceiling          0        -
```

Completion rate was 100%. Every run passed on the first try, because the Planner output already gets cleaned into 3 tags and a short summary before validation.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL JUPYTER TRUFFLE

• (data260) villyalsriprajee@Mac data260-3319 % python -u code/experiments_hw02.py --experiment schema 2>&1 | tee -a reports/hw02/RUN_LOG.txt
[schema30] run 1/30: valid_first_attempt (retries=0, 209854 ms)
[schema30] run 2/30: valid_first_attempt (retries=0, 117006 ms)
[schema30] run 3/30: valid_first_attempt (retries=0, 109812 ms)
[schema30] run 4/30: valid_first_attempt (retries=0, 95399 ms)
[schema30] run 5/30: valid_first_attempt (retries=0, 95721 ms)
[schema30] run 6/30: valid_first_attempt (retries=0, 155636 ms)
[schema30] run 7/30: valid_first_attempt (retries=0, 139496 ms)
[schema30] run 8/30: valid_first_attempt (retries=0, 173950 ms)
[schema30] run 9/30: valid_first_attempt (retries=0, 116821 ms)
[schema30] run 10/30: valid_first_attempt (retries=0, 115709 ms)
[schema30] run 11/30: valid_first_attempt (retries=0, 111057 ms)
[schema30] run 12/30: valid_first_attempt (retries=0, 104937 ms)
[schema30] run 13/30: valid_first_attempt (retries=0, 99060 ms)
[schema30] run 14/30: valid_first_attempt (retries=0, 94454 ms)
[schema30] run 15/30: valid_first_attempt (retries=0, 103251 ms)
[schema30] run 16/30: valid_first_attempt (retries=0, 121355 ms)
[schema30] run 17/30: valid_first_attempt (retries=0, 122042 ms)
[schema30] run 18/30: valid_first_attempt (retries=0, 131502 ms)
[schema30] run 19/30: valid_first_attempt (retries=0, 142131 ms)
[schema30] run 20/30: valid_first_attempt (retries=0, 96738 ms)
[schema30] run 21/30: valid_first_attempt (retries=0, 94585 ms)
[schema30] run 22/30: valid_first_attempt (retries=0, 99522 ms)
[schema30] run 23/30: valid_first_attempt (retries=0, 95385 ms)
[schema30] run 24/30: valid_first_attempt (retries=0, 104986 ms)
[schema30] run 25/30: valid_first_attempt (retries=0, 110167 ms)
[schema30] run 26/30: valid_first_attempt (retries=0, 118099 ms)
[schema30] run 27/30: valid_first_attempt (retries=0, 127071 ms)
[schema30] run 28/30: valid_first_attempt (retries=0, 121165 ms)
[schema30] run 29/30: valid_first_attempt (retries=0, 96928 ms)
[schema30] run 30/30: valid_first_attempt (retries=0, 98763 ms)
wrote schema_runs.csv and schema_runs.json

=== schema30 summary ===
runs: 30
completion rate: 100.00%
mean latency: 117420 ms
valid_first_attempt: 30
valid_after_1_retry: 0
valid_after_2plus_retries: 0
hit_ceiling: 0
(data260) villyalsriprajee@Mac data260-3319 %
```

Experiment 2 - ceiling 2 vs 10

Same input, 20 runs at ceiling 2 and 20 at ceiling 10.

Ceiling comparison (2 vs 10)			
Ceiling	Runs	Completion	Mean latency (ms)
2	20	100%	102658
10	20	100%	101082

Both finished every run and the latencies are about the same. I'd pick ceiling 2 for deployment since it limits retries in a bad case without hurting the good ones.

```
(data260) villyalsriprajeet@Mac data260-3319 % python -u code/experiments_hw02.py --experiment ceiling10 2>&1 | tee -a reports/hw02/RUN_
[ceiling10] run 1/20: valid_first_attempt (retries=0, 96004 ms)
[ceiling10] run 2/20: valid_first_attempt (retries=0, 97086 ms)
[ceiling10] run 3/20: valid_first_attempt (retries=0, 99963 ms)
[ceiling10] run 4/20: valid_first_attempt (retries=0, 101642 ms)
[ceiling10] run 5/20: valid_first_attempt (retries=0, 101217 ms)
[ceiling10] run 6/20: valid_first_attempt (retries=0, 99923 ms)
[ceiling10] run 7/20: valid_first_attempt (retries=0, 93697 ms)
[ceiling10] run 8/20: valid_first_attempt (retries=0, 94453 ms)
[ceiling10] run 9/20: valid_first_attempt (retries=0, 93654 ms)
[ceiling10] run 10/20: valid_first_attempt (retries=0, 117264 ms)
[ceiling10] run 11/20: valid_first_attempt (retries=0, 115940 ms)
[ceiling10] run 12/20: valid_first_attempt (retries=0, 93756 ms)
[ceiling10] run 13/20: valid_first_attempt (retries=0, 93645 ms)
[ceiling10] run 14/20: valid_first_attempt (retries=0, 104437 ms)
[ceiling10] run 15/20: valid_first_attempt (retries=0, 98905 ms)
[ceiling10] run 16/20: valid_first_attempt (retries=0, 98654 ms)
[ceiling10] run 17/20: valid_first_attempt (retries=0, 106250 ms)
[ceiling10] run 18/20: valid_first_attempt (retries=0, 103053 ms)
[ceiling10] run 19/20: valid_first_attempt (retries=0, 102543 ms)
[ceiling10] run 20/20: valid_first_attempt (retries=0, 109565 ms)
wrote ceiling10_runs.csv and ceiling10_runs.json

=== ceiling10 summary ===
runs: 20
completion rate: 100.00%
mean latency: 101082 ms
valid_first_attempt: 20
valid_after_1_retry: 0
valid_after_2plus_retries: 0
hit_ceiling: 0
(data260) villyalsriprajeet@Mac data260-3319 %
• (data260) villyalsriprajeet@Mac data260-3319 % python -u code/experiments_hw02.py --experiment ceiling2 2>&1 | tee -a reports/hw02/RUN_LOG.txt
[ceiling2] run 1/20: valid_first_attempt (retries=0, 95388 ms)
[ceiling2] run 2/20: valid_first_attempt (retries=0, 95619 ms)
[ceiling2] run 3/20: valid_first_attempt (retries=0, 98513 ms)
[ceiling2] run 4/20: valid_first_attempt (retries=0, 95595 ms)
[ceiling2] run 5/20: valid_first_attempt (retries=0, 94634 ms)
[ceiling2] run 6/20: valid_first_attempt (retries=0, 101438 ms)
[ceiling2] run 7/20: valid_first_attempt (retries=0, 107434 ms)
[ceiling2] run 8/20: valid_first_attempt (retries=0, 109560 ms)
[ceiling2] run 9/20: valid_first_attempt (retries=0, 108359 ms)
[ceiling2] run 10/20: valid_first_attempt (retries=0, 110907 ms)
[ceiling2] run 11/20: valid_first_attempt (retries=0, 112548 ms)
[ceiling2] run 12/20: valid_first_attempt (retries=0, 93548 ms)
[ceiling2] run 13/20: valid_first_attempt (retries=0, 98017 ms)
[ceiling2] run 14/20: valid_first_attempt (retries=0, 106211 ms)
[ceiling2] run 15/20: valid_first_attempt (retries=0, 95895 ms)
[ceiling2] run 16/20: valid_first_attempt (retries=0, 93465 ms)
[ceiling2] run 17/20: valid_first_attempt (retries=0, 94807 ms)
[ceiling2] run 18/20: valid_first_attempt (retries=0, 107026 ms)
[ceiling2] run 19/20: valid_first_attempt (retries=0, 112601 ms)
[ceiling2] run 20/20: valid_first_attempt (retries=0, 121600 ms)
wrote ceiling2_runs.csv and ceiling2_runs.json

=== ceiling2 summary ===
runs: 20
completion rate: 100.00%
mean latency: 102658 ms
valid_first_attempt: 20
valid_after_1_retry: 0
valid_after_2plus_retries: 0
hit_ceiling: 0
(data260) villyalsriprajeet@Mac data260-3319 %
```

Experiment 3 - adversarial input

I made an adversarial input that breaks all three rules at once, a junk 46-character title, an injected "IGNORE ALL PRIOR INSTRUCTIONS" line, and no real content to tag and ran it 5 times.

```
Experiment 3 - adversarial input (5 runs)
runs: 5   completion rate: 100.00%   overall mean latency: 201963 ms

Outcome over 5 runs      Count   Mean latency (ms)
Valid first attempt      1      119391
Valid after 1 retry      4      222606
Valid after 2+ retries   0      -
Hit turn ceiling         0      -
```

Completion was 100%, but 4 of the 5 runs needed a retry. On the first attempt the Planner usually failed validation, then the retry fed the error back and it fixed it. It didn't hit the ceiling in 4 or more runs. The fix I'd propose is to clean the input before it reaches the Planner, strip injected instructions, cap the title length, and reject empty content, so bad input never becomes a real task.

```
• (data260) villyalsriprajeet@Mac data260-3319 % python -u code/experiments_hw02.py --experiment adversarial 2>&1 | tee -a reports/hw02/RUN_LOG.txt
[adversarial] run 1/5: valid_first_attempt (retries=0, 119391 ms)
[adversarial] run 2/5: valid_after_1_retry (retries=1, 207964 ms)
[adversarial] run 3/5: valid_after_1_retry (retries=1, 242230 ms)
[adversarial] run 4/5: valid_after_1_retry (retries=1, 211835 ms)
[adversarial] run 5/5: valid_after_1_retry (retries=1, 228396 ms)
wrote adversarial_runs.csv and adversarial_runs.json

=== adversarial summary ===
runs: 5
completion rate: 100.00%
mean latency: 201963 ms
valid_first_attempt: 1
valid_after_1_retry: 4
valid_after_2plus_retries: 0
hit_ceiling: 0
• (data260) villyalsriprajeet@Mac data260-3319 %
```

Verification

I wrote a smoke test (code/verify_hw02.py, run with make verify-hw02). The schema accepts a valid 3-tag output and rejects bad ones, the LangGraph graph compiles, and the FastAPI backend responds on port 8619. All checks passed.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER: VARIABLES
• (data260) villyalsriprajeet@Mac data260-3319 % make verify-hw02
python code/verify_hw02.py
[PASS] agents_graph.py exists
[PASS] schema_hw02.py exists
[PASS] runner_hw02.py exists
[PASS] FastAPI main.py exists
[PASS] schema_input.json exists
[PASS] schema accepts a valid 3-tag output
[PASS] schema rejects wrong tag count
[PASS] schema rejects out-of-range tag length
[PASS] LangGraph graph compiles
[PASS] FastAPI responds 200 on port 8619

{
  "homework": "Hw2",
  "SID4": 3319,
  "commit_hash": "9eb27b80171d1a5a8e530f79ecdfca2240c28b7e",
  "model": "qwen2.5:3b",
  "SEED": 3319,
  "VERIFY_SEED": 263319,
  "all_passed": true,
  "checks": [
    {
      "check": "agents_graph.py exists",
      "passed": true
    },
    {
      "check": "schema_hw02.py exists",
      "passed": true
    },
    {
      "check": "runner_hw02.py exists",
      "passed": true
    },
    {
      "check": "FastAPI main.py exists",
      "passed": true
    },
    {
      "check": "schema_input.json exists",
      "passed": true
    },
    {
      "check": "schema accepts a valid 3-tag output",
      "passed": true
    },
    {
      "check": "schema rejects wrong tag count",
      "passed": true
    },
    {
      "check": "schema rejects out-of-range tag length",
      "passed": true
    },
    {
      "check": "LangGraph graph compiles",
      "passed": true
    },
    {
      "check": "FastAPI responds 200 on port 8619",
      "passed": true
    }
  ]
}

Wrote /Users/villyalsriprajeet/Desktop/data260-3319/reports/hw02/verification.json
• (data260) villyalsriprajeet@Mac data260-3319 %
```

Summary

I built HW2 on my HW1 project, and extended it across all four parts. In Part 1 I made the fixtures page usable at 375px and added visible loading, empty and error states. In Part 2 I added a FastAPI backend on port 8619 that can add, update, delete and search records, redirecting back to the home view after each action. In Part 3 I refactored my HW1 Planner-Reviewer script into a stateful LangGraph graph with a supervisor and router, so it can route work and loop back to self correct, with all LLM calls going through `model_client.py`. In Part 4 I added a Pydantic schema to validate the Planner output, with a retry loop that feeds the error back on failure, and I ran three experiments on it, a 30 run schema test, a ceiling comparison, and an adversarial input. Everything ran locally on qwen2.5:3b through Ollama, and a smoke test confirms the schema, the graph and the FastAPI backend all still work.